

Project Checkpoint 2

Vetrivel Maheswaran
vm6923@rit.edu

I. RECENT PROGRESS

The research question driving this project is whether combining vector retrieval with graph-based retrieval reduces hallucinated and unsupported answers compared to a vector-only baseline [1], [3]. Since Checkpoint 1, progress has been made across the document processing pipeline, hybrid retrieval, the system interface, and a formal vector-mode evaluation that establishes the baseline metrics for the upcoming hybrid comparison.

A. Document Extraction and Cleaning

The extraction stage converts uploaded PDF and DOCX files into clean, structured text. For PDFs, three tools are combined—pdfplumber, Pix2Text for mathematical content, and Tesseract OCR as a fallback because scientific documents contain equations and multi-column layouts that a single parser cannot handle reliably. DOCX files are parsed directly from their XML structure to preserve heading hierarchy. A cleaning step then normalises the text while preserving formula and figure markers, since removing structured content would reduce the evidence available for retrieval.

B. Semantic Chunking

Chunking determines the units the retriever returns as evidence. Poor boundaries produce incomplete evidence, which is a direct cause of unsupported answers [3]. The chunker was redesigned to be section-first and token-aware, respecting document structure before applying size limits. Blocks are classified in priority order:

- Reference entries are isolated and excluded from the index, since bibliography text is not a useful retrieval target.
- Formula and table blocks are kept atomic and never split across chunk boundaries.
- Figure captions are marked non-indexable because they lack standalone retrievable content.
- List items are paired with their description paragraph so each chunk is self-contained.
- Section headings are merged with the following body paragraph so every chunk carries its structural context.
- Remaining paragraphs are accumulated into token-bounded chunks.

Section context is also carried across page boundaries, fixing a bug where continuation paragraphs inherited the wrong section label. Every chunk includes an `embedding_text` field that prepends the document name and section heading, so the vector encodes where the chunk comes from as well as what it says. This improves retrieval precision for section-specific queries.

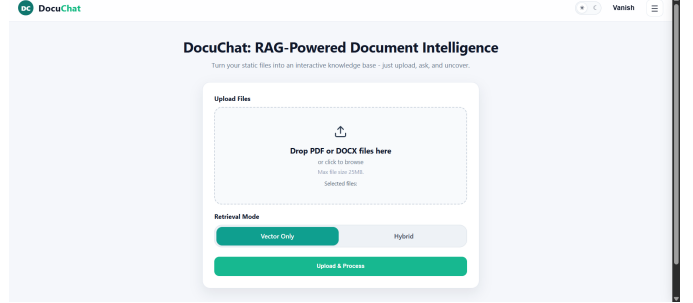


Fig. 1. Landing page UI

C. Embedding and Vector Indexing

Each chunk is embedded using `text-embedding-3-small` on its `embedding_text` (document and section prefix included), so the vector encodes structural context alongside semantic content. A SHA-256 cache avoids re-embedding unchanged chunks across repeated uploads, reducing API cost and processing time.

Embeddings are stored in a FAISS cosine similarity index [1]. Each indexed chunk is paired with a metadata record storing the chunk and document identifiers, page number, character offsets, section path, and chunk type. This metadata enables the system to produce attributed, citable answers that point back to specific document locations—which is essential for grounding analysis.

D. System Interface and Logging

To make the system behaviour observable and easier to analyse, the user interface was improved and query logging was added.

Fig. 1 shows the landing page where users can upload one or more documents and select a retrieval mode before processing.

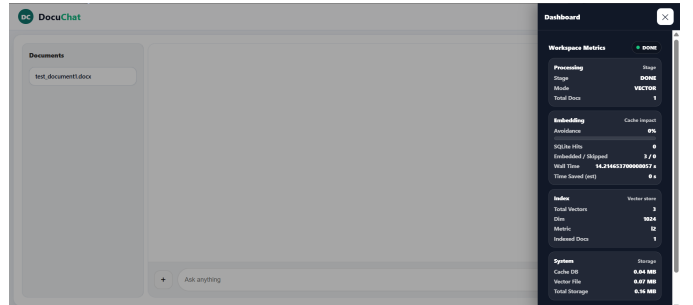


Fig. 2. Chat UI for vector retrieval pipeline with system metrics.

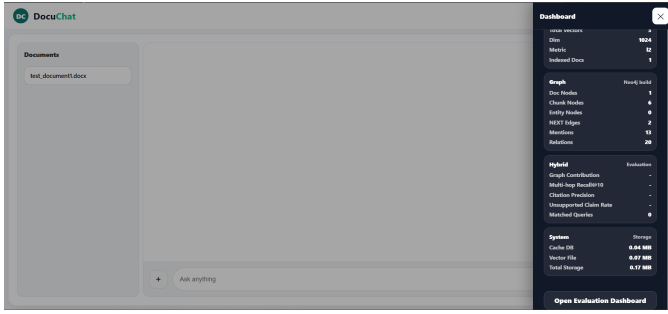


Fig. 3. Chat UI for hybrid retrieval pipeline with graph and hybrid metrics.

Since the project compares vector-only and hybrid retrieval, the interface supports controlled mode selection.

Fig. 2 shows the chat interface for the vector retrieval pipeline. It is included to demonstrate how processing details and system metrics are exposed in the sidebar so that retrieval behaviour is more transparent.

Fig. 3 shows the chat interface for the hybrid retrieval pipeline, including additional graph and hybrid-related information in the sidebar. This figure is important because it demonstrates that graph-enhanced retrieval is not only implemented internally, but is also visible as a separate retrieval mode with its own supporting statistics.

In addition, query logging was implemented to store each query, retrieved chunks, citations, and generated answers. This is necessary for later evaluation because the metric pipeline depends on these logs to analyse retrieval quality, grounding quality, and answer correctness.

E. Grounded Answer Generation

The generation stage restricts the language model to producing answers only from the retrieved chunks in the prompt. If the evidence is insufficient, the system outputs the fixed string “*Not found in documents.*” rather than speculating. This abstention behaviour is central to the research question: a system that refuses to answer when evidence is absent cannot hallucinate on those queries [5]. Each source chunk is formatted with its document name, section path, and page number so the model cites specific locations rather than producing anonymous statements.

F. Evaluation Framework and Vector Baseline

The evaluation framework was built to answer the research question quantitatively by separating distinct failure modes [4], [6]: Recall@ k isolates retrieval failure; Citation Precision isolates grounding failure; Abstention Accuracy and Unsupported Claim Rate isolate hallucination; Answer F1 measures end-to-end quality. Separating these failure modes is necessary to determine whether any improvement from hybrid retrieval comes from better evidence coverage or from some other factor.

a) Evaluation Dataset: To evaluate the pipeline, I selected three documents of different types and formats: a two-column IEEE-style PDF, a structured DOCX proposal, and a short assignment PDF. Together these test the extraction and chunking pipeline across plain text, tables, and structured layouts. I

Metric	Score	Query Count
Recall@10	92.9%	56
Abstention Accuracy	95.1%	82
Citation Precision	51.8%	82
Unsupported Claim Rate	23.2%	82
Answer F1	54.5%	56
Answer Exact Match	0.0%	56
Graph Contribution	N/A	Hybrid only
Multihop Recall@10	N/A	Hybrid only

Table 1. Vector-Mode Baseline Evaluation Results ($k=10$)

manually created 83 queries covering factual, multi-chunk (8 queries), cross-document (4 queries), and unanswerable queries (26 queries, 31.3% of total), each with a reference chunk ID and notes. Unanswerable queries are included to directly measure whether the system abstains correctly instead of hallucinating.

b) Vector Baseline Results: The vector-mode evaluation was executed against all 83 queries. Table 1 presents the results and serves as the baseline that all hybrid evaluation results in Checkpoint 4 will be compared against. This comparison will determine whether the graph layer reduces unsupported answers.

Recall@10 at 92.9% shows the retriever finds the correct evidence in the top 10 results for the majority of queries. The remaining gap is structural: two queries target non-indexed cover page content, and two multi-chunk queries retrieved only one of two required chunks.

Abstention Accuracy at 95.1% shows strong hallucination control. The system correctly refused to answer 22 of 26 unanswerable queries. Three failures occurred because the answer was on a non-indexed page. One genuine hallucination was identified: the system answered using pre-training knowledge rather than abstaining when the answer was not in the indexed content [5].

Citation Precision at 51.8% is the primary area for improvement before the hybrid evaluation. The cause is identified: three chunks are passed to the model at generation time, and the model cites all three even when only one is directly relevant. Unsupported Claim Rate at 23.2% is a direct consequence of the same behaviour. Both will be addressed by reducing the context window before the hybrid run.

Answer F1 at 54.5% is reasonable: short exact-answer queries score above 0.90, while synthesis queries score 0.30–0.45 because the model paraphrases correctly but differently from the gold string. Exact Match at 0.0% is expected for any generative model and does not indicate a failure.

G. Graph Construction and Hybrid Retrieval

Vector similarity search operates at the individual chunk level and may miss evidence distributed across related chunks [2], [7]. This incomplete coverage is a direct cause of unsupported answers, which is the core problem the research question targets.

A Neo4j knowledge graph was built from document chunks using three node types (document, chunk, entity) and two edge

types (adjacency between sequential chunks and semantic edges through shared entities). Entity extraction uses a language-model-assisted approach because it adapts to each document’s vocabulary without requiring domain-specific rules.

The hybrid retrieval pipeline has been implemented and is ready for evaluation: it runs vector search and graph traversal in parallel, merges and deduplicates results, and passes a unified context to the language model. Two additional metrics were designed for the hybrid evaluation: Graph Contribution Rate measures what fraction of the final context came from the graph channel, and Multihop Recall@k measures recall on queries requiring multiple supporting chunks. The hybrid evaluation run and comparison against the vector baseline are the primary goal of next Checkpoint.

II. QUESTIONS / CONCERNS

- 1) **Evaluation dashboard scope.** My current plan includes summary metric cards comparing vector versus hybrid, a per-query breakdown table, and Graph Contribution Rate and Multihop Recall visualisations for hybrid mode. If breakdowns by query type or document source are also expected, please clarify so I can plan the scope correctly.

III. NEXT STEPS

- 1) **Fix Citation Precision.** Reduce the context window from 3 to 2 chunks and update the citation instruction in the prompt. Re-run the vector evaluation to confirm improvement.
- 2) **Run hybrid evaluation.** Execute all queries in hybrid mode and produce the vector-versus-hybrid metric comparison. This is the central result needed to answer the research question.
- 3) **Analyse and interpret the comparison.** Determine whether the graph layer improves Recall@k and reduces Unsupported Claim Rate, and quantify Graph Contribution Rate.
- 4) **Build the evaluation dashboard.** Implement the dashboard displaying metric summaries, vector-versus-hybrid comparison, and per-query results.
- 5) **Conduct error analysis.** Categorise failures from the per-query output as retrieval failure, grounding failure, or generation failure to support the final report discussion.
- 6) **Prepare the final report.** Connect all design decisions to the research question and summarise what the metric comparison reveals about hybrid versus vector-only retrieval.

REFERENCES

- [1] K. Sawarkar, A. Mangal and S. R. Solanki, “Blended RAG: Improving RAG Accuracy with Semantic Search and Hybrid Query-Based Retrievers,” *2024 IEEE 7th Int. Conf. on Multimedia Information Processing and Retrieval (MIPR)*, 2024, pp. 155–161.
- [2] B. Sarmah et al., “HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction,” *arXiv:2408.04948*, August 2024.
- [3] R. Koç, M. K. Gürkan and F. T. Yarman Vural, “ReRag: A New Architecture for Reducing Hallucination by Retrieval-Augmented Generation,” *2024 9th Int. Conf. on Computer Science and Engineering (UBMK)*, 2024, pp. 961–965.
- [4] H. Hu, C. He, X. Xie and Q. Zhang, “LRP4RAG: Detecting Hallucinations in Retrieval-Augmented Generation via Layer-Wise Relevance Propagation,” *arXiv:2408.15533*, June 27, 2025.
- [5] Y. A. Yadkori et al., “Mitigating LLM Hallucinations via Conformal Abstention,” *arXiv:2405.01563*, April 2024.
- [6] W. T. Chan et al., “Optimizing Confidence Scoring in RAG-Based LLM Chatbots for Technical Support Services: A Prompt Engineering Approach,” *2025 IEEE 15th Symp. on Computer Applications & Industrial Electronics (ISCAIE)*, 2025, pp. 540–545.
- [7] S. Knollmeyer et al., “Document Knowledge Graph to Enhance Question Answering with Retrieval Augmented Generation,” *2024 IEEE 29th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2024, pp. 1–4.